Step Contracts and State Handling

MSA 8770 Milestone 2 – Insurance Claims Intake Triage (Team 7)

Section owner: **Tejaswini Waghmare**

The prototype runs each claim through a fixed sequence of ten steps (S1–S10) and two simulated human steps (H1, H2). Every step reads from and writes to one shared state object, which is saved to disk before each step so a stopped claim can be resumed. This section documents each step's contract and how the implementation stores state, handles failures, logs execution and resumes after human review. All behaviour described here is taken from `src/state.py` and `src/pipeline.py`.

1. Workflow Step Contracts

Each step stores its output in `state["results"]["<step>"]`. A step returns nothing to continue, or a terminal status to stop the run.

Step	Input	Operation	Output (state results)	Validation and failure behaviour
S1 Input check (Code)	Claim form + narrative	Confirm the 5 required fields (claim ID, policy number, incident date, claimed amount, narrative) are present	`missing_fields` list	Any field missing → `PAUSED_MISSING_INFO`. No retry; waits for customer.
S2 Policy retrieval (Tool)	Policy number, narrative	Look up policy record; get CSL tier; keyword-rank policy clauses	Tier, limit, deductible, report threshold, bind date, annual premium, top 3 clauses	Policy not found → `FAILED`. No retry.
S3 Extraction (LLM)	Narrative, prompt version (B)	Extract JSON: incident date, type, amount, police report, vehicle, summary	`extracted` fields + `attempts` history	API error/timeout → up to 3 attempts (1s, 2s backoff), then `ROUTED_TO_HUMAN`.
S4 Validation (Code)	S3 output, form fields	Schema check (Pydantic); cross-check date and amount (±5%) against form	`issues` list	Invalid output → re-prompt with the error (3 extraction attempts in total), then `ROUTED_TO_HUMAN`. Mismatch → `ROUTED_TO_HUMAN`.
S5 Coverage (Rules)	Validated fields, S2 policy	Check bind date, policy limit, police report threshold; calculate payout	`COVERED` / `NOT_COVERED` / `AMBIGUOUS`, reasons, payout	Not covered or ambiguous → `ROUTED_TO_HUMAN`. Deterministic; no retry.
S6 Features (Code)	Claims record + validated fields	Build 14 numeric + 6 categorical features (includes customer age; no other demographic fields)	`features_built`; full record kept in memory only (`_record`)	Unexpected error → `FAILED`, state saved for resume.
S7 Risk score (ML)	S6 feature row	Logistic regression fraud probability + rule-based flags	Score, method, rule score, flags	Model error → rule-based score (`rule_based_fallback`). No repeated ML attempts.
S8 Score check (Code)	S7 output	Verify score is a number in [0, 1]; assign band (LOW < 0.4 ≤ MEDIUM < 0.7 ≤ HIGH)	`valid`, `band`	Invalid → rule score; still invalid → `ROUTED_TO_HUMAN`.
S9 Explanation (LLM)	Summary, coverage, score, flags	Generate 2–3 neutral sentences explaining the risk for the adjuster	Explanation text + `source` (llm or template_fallback)	API failure or empty text → template explanation; run continues.
S10 Package (Code)	All prior results	Write review package and suggested next action	`outputs/<claim_id>_package.json`	Sets status to `COMPLETED_PENDING_ADJUSTER`.
H1 Exception	Saved state + reviewer input	Add missing data, correct extracted fields, confirm	Entry in `human_actions`	Resumes the run at S1, S3 (stopped step), S5 or S6.

Step	Input	Operation	Output (state results)	Validation and failure behaviour
review (Human, simulated)		coverage, or retry		
H2 Adjuster decision (Human, simulated)	State at `COMPLETED_PENDING_ADJUSTER`	Approve or deny the claim	`APPROVED` / `DENIED`; payment and notification mocked	Rejected if the claim is not ready for decision.

2. Workflow State

`new_state()` creates one Python dictionary per claim. The table lists its fields.

Field	Contents
`run_id`, `claim_id`	Unique run identifier and the claim being processed
`status`, `current_step`, `status_reason`	Where the run is, the step being executed or retried, and a readable reason when the run stops
`input`	The original claim submission (form fields + narrative)
`config`	Prompt version, model name and test-failure flags
`results`	Intermediate output of every step, keyed by step ID (S1–S10)
`errors`	List of failures; each has the step, attempt number, error type (api_error, invalid_output, model_error, crash), message and timestamp
`retries`	Retry counters by step and kind: `S3_api`, `S3_reprompt`, `S9_api`
`human_actions`	Reviewer, action taken, note, previous status and timestamp for every H1/H2 action
`metrics`	LLM call count, LLM latency, prompt and output tokens, total pipeline time
`started_at`, `updated_at`	Run timestamps

Status lifecycle: `RUNNING` → `PAUSED_MISSING_INFO` | `ROUTED_TO_HUMAN` | `FAILED` | `COMPLETED_PENDING_ADJUSTER` → `APPROVED` | `DENIED`.

3. Persistence and Checkpointing

- **Checkpoint before each step.** The orchestrator `run_from()` sets `current_step` and saves the state to `states/<claim_id>.json` before running each step, and once more when the run ends. If a run stops or crashes, `current_step` therefore names the step to retry.
- **Crash handling.** An unexpected exception is caught, logged as `crash`, and sets the status to `FAILED`; the saved state is kept for investigation or resume.
- **Privacy.** `save_state()` skips keys that start with `_`. The full customer record built in S6 (`_record`) is kept in memory only, never written to disk, and rebuilt when a run resumes at S7 or later.

4. Retry and Failure Handling

Failure	Handling	Limit	Recorded in state
LLM API error or timeout (S3, S9)	Retry with exponential backoff (1s, 2s); S3 then routes to human, S9 falls back to template text	3 attempts	`errors` (api_error), `retries.S3_api` / `S9_api`
Invalid extraction output (S3/S4)	Re-prompt including the validation error; then route to human review	3 extraction attempts (1 + 2 re-prompts)	`errors` (invalid_output), `retries.S3_reprompt`

Failure	Handling	Limit	Recorded in state
ML risk model error (S7)	Use rule-based score instead of retrying the model	1 attempt	`results.S7.method = rule_based_fallback`
Invalid risk score (S8)	Switch to rule score; route to human if still invalid	1 fallback	`results.S8.valid`
Missing input (S1)	Pause and wait for customer	None	`status = PAUSED_MISSING_INFO`
Policy not found (S2)	Stop the run	None	`status = FAILED`
Rule/code steps (S4, S5)	Deterministic; route to human on mismatch or unclear coverage	None	`status = ROUTED_TO_HUMAN`, `status_reason`

5. Human Intervention and Resume

`resume_claim()` loads the saved state, clears the test-failure flags, records the reviewer action in `human_actions`, logs an `H1` event and restarts from the step that matches the action.

Situation	Reviewer action	Resumes at	Test evidence
Missing information (S1)	Customer supplies fields (`new_input`)	S1 (full re-validation)	TC2 → completed
Invalid model output or outage (S3)	Plain retry of the stopped step	`current_step` (S3)	TC4 → completed
Form vs narrative mismatch (S4)	Reviewer corrects fields (`corrected_fields`); S4 marked `overridden_by`	S5	TC5 → completed
Ambiguous coverage (S5)	Reviewer confirms coverage (`override_coverage`)	S6	TC3 → completed

Final human gate. The pipeline stops at `COMPLETED_PENDING_ADJUSTER`; the package records that a human adjuster must approve or deny. `adjuster_decision()` only accepts claims in that status and sets `APPROVED` or `DENIED`. Payment and notification are mocked as log entries.

6. Logging and Observability

`log_step()` appends one JSON line per event to `logs/run_log.jsonl` and prints it to the console. Each line records the timestamp, run ID, claim ID, step, status (ok, fail, invalid, api_error, fallback, crash, human, mock), attempt number, latency in milliseconds and a detail message (up to 300 characters). Every run ends with an `END` line holding the final status. Together with the `metrics` field in the state, this gives per-step evidence for reliability, latency and token cost.

7. Known Limitations

- Retry counters are not reset on resume, so limits apply per run rather than over the claim's lifetime (TC4 still shows `S3_reprompt: 3` after it completes).
- Fields corrected by a reviewer skip the S4 cross-check and are not re-validated.
- `resume_claim()` does not check the claim's current status before resuming.
- S9 only checks that the explanation is not empty; it does not check that the text is grounded in the claim data, and the model can add details not found in the inputs.

8. Evidence

src/state.py

```python
def new_state(claim):
    """Create the state dictionary that is carried between all steps."""
    return {
        "run_id": uuid.uuid4().hex[:8],
        "claim_id": claim["claim_id"],
        "status": "RUNNING",          # RUNNING | PAUSED_MISSING_INFO | ROUTED_TO_HUMAN | FAILED
                                      # | COMPLETED_PENDING_ADJUSTER | APPROVED | DENIED
        "current_step": "S1",
        "status_reason": None,
        "input": claim,
        "config": {},
        "results": {},                # intermediate output of every step, keyed by step id
        "errors": [],                 # every error with step, attempt and message
        "retries": {},                # retry counters per step
        "human_actions": [],          # reviewer corrections / overrides / decisions
        "metrics": {"llm_calls": 0, "llm_latency_ms": 0, "prompt_tokens": 0, "output_tokens": 0},
        "started_at": now(),
        "updated_at": now(),
    }
```

Figure A. Workflow state definition. The state tracks the claim, current step, intermediate results, errors, retry counters, human actions and metrics.

src/state.py

```python
def save_state(state):
    """Persist the full state after every step so a run can be resumed or audited."""
    os.makedirs(STATE_DIR, exist_ok=True)
    state["updated_at"] = now()
    to_save = {k: v for k, v in state.items() if not k.startswith("_")}   # skip in-memory-only items
    with open(os.path.join(STATE_DIR, f"{state['claim_id']}.json"), "w", encoding="utf-8") as f:
        json.dump(to_save, f, indent=2, default=str)


def load_state(claim_id):
    with open(os.path.join(STATE_DIR, f"{claim_id}.json"), encoding="utf-8") as f:
        return json.load(f)


def log_step(state, step, status, detail="", attempt=1, latency_ms=None):
    """Append one line per event to logs/run_log.jsonl and echo it to the console."""
    os.makedirs(os.path.dirname(LOG_PATH), exist_ok=True)
    rec = {"ts": now(), "run_id": state["run_id"], "claim_id": state["claim_id"], "step": step,
           "status": status, "attempt": attempt, "latency_ms": latency_ms, "detail": str(detail)[:300]}
    with open(LOG_PATH, "a", encoding="utf-8") as f:
        f.write(json.dumps(rec) + "\n")
```

Figure B. State persistence and logging. State is written to `states/<claim_id>.json` and each step event to `logs/run_log.jsonl`; keys starting with _ are not saved.

src/pipeline.py

```python
def run_from(state, start_step):
    """Runs the fixed sequence from start_step until a step returns a terminal status."""
    t0 = time.time()
    state["status"] = "RUNNING"
    for step in STEP_ORDER[STEP_ORDER.index(start_step):]:
        state["current_step"] = step
        save_state(state)                       # checkpoint before each step (supports resume)
        if step in ("S7", "S8", "S9", "S10") and "_record" not in state:
            s6_features(state)                  # rebuild in-memory features after a resume
        try:
            outcome = STEPS[step](state)
        except Exception as e:                  # unexpected crash -> FAILED, state kept for resume
            state["errors"].append({"step": step, "attempt": 1, "type": "crash", "error": repr(e), "ts": now()})
            state["status_reason"] = f"Unexpected error in {step}: {e}"
            log_step(state, step, "crash", e)
            outcome = "FAILED"
        if outcome:
            state["status"] = outcome
            break
    state["metrics"]["pipeline_ms"] = state["metrics"].get("pipeline_ms", 0) + int((time.time() - t0) * 1000)
    state.pop("_record", None)
    save_state(state)
    log_step(state, "END", state["status"], state["status_reason"])
    return state
```

Figure C. Orchestrator checkpointing. State is saved before each step, so a stopped run can resume at `current_step`.

src/pipeline.py

```python
MAX_API_ATTEMPTS = 3          # LLM call errors / timeouts: 3 attempts, backoff 1s, 2s
MAX_EXTRACTION_ATTEMPTS = 3   # invalid JSON / schema: first try + 2 re-prompts with the error message
AMOUNT_TOLERANCE = 0.05       # narrative vs form amount may differ by at most 5%
REQUIRED_FIELDS = ["claim_id", "policy_number", "form_incident_date", "form_claimed_amount", "narrative"]
```

...

```python
def call_llm_with_retry(state, step, prompt, json_mode=False):
    """Calls the model; on API errors retries up to MAX_API_ATTEMPTS with exponential backoff."""
    cfg = state["config"]
    for attempt in range(1, MAX_API_ATTEMPTS + 1):
        try:
            if cfg.get("force_api_fail"):
                raise ConnectionError("simulated LLM API outage")
            meta = llm_with_meta(prompt, model=cfg.get("model"), json_mode=json_mode)
            m = state["metrics"]
            m["llm_calls"] += 1
            m["llm_latency_ms"] += meta["latency_ms"]
            m["prompt_tokens"] += meta["prompt_tokens"]
            m["output_tokens"] += meta["output_tokens"]
            return meta
        except Exception as e:
            state["errors"].append({"step": step, "attempt": attempt, "type": "api_error", "error": str(e), "ts": now()})
            state["retries"][f"{step}_api"] = attempt
            log_step(state, step, "api_error", e, attempt)
            if attempt < MAX_API_ATTEMPTS:
                time.sleep(2 ** (attempt - 1))
    raise LLMUnavailable(f"LLM unavailable after {MAX_API_ATTEMPTS} attempts")
```

src/pipeline.py

```python
def s3_extract(state):
    """S3 model call + S4 schema validation inside a bounded re-prompt loop."""
    cfg = state["config"]
    feedback, attempts = None, []
    for attempt in range(1, MAX_EXTRACTION_ATTEMPTS + 1):
        prompt = build_prompt(state["input"]["narrative"], cfg.get("prompt_version", "B"), feedback)
        try:
            if cfg.get("force_bad_json"):
                meta = {"text": "Sorry, I could not process this claim.", "latency_ms": 0}  # simulated bad output
            else:
                meta = call_llm_with_retry(state, "S3", prompt, json_mode=cfg.get("prompt_version", "B") == "B")
        except LLMUnavailable as e:
            state["status_reason"] = f"{e}. Routed to human exception review (H1)."
            log_step(state, "S3", "fail", state["status_reason"], attempt)
            return "ROUTED_TO_HUMAN"
        parsed, error = parse_and_validate(meta["text"])
        attempts.append({"attempt": attempt, "raw": meta["text"][:500], "valid": parsed is not None, "error": error})
        if parsed:
            state["results"]["S3"] = {"extracted": parsed, "attempts": attempts}
            log_step(state, "S3", "ok", parsed, attempt, meta["latency_ms"])
            return None
        state["retries"]["S3_reprompt"] = attempt
        state["errors"].append({"step": "S4", "attempt": attempt, "type": "invalid_output", "error": error, "ts": now()})
        log_step(state, "S4", "invalid", error, attempt, meta["latency_ms"])
        feedback = error
    state["results"]["S3"] = {"extracted": None, "attempts": attempts}
    state["status_reason"] = (f"Model output invalid after {MAX_EXTRACTION_ATTEMPTS} attempts. "
                              "Routed to human exception review (H1).")
    log_step(state, "S3", "fail", state["status_reason"], MAX_EXTRACTION_ATTEMPTS)
    return "ROUTED_TO_HUMAN"
```

Figure D. Bounded retries. LLM calls are limited to 3 attempts with backoff; invalid extraction output is re-prompted up to 3 attempts before human review.

src/pipeline.py

```python
def resume_claim(claim_id, reviewer, note="", corrected_fields=None, new_input=None,
                 override_coverage=False, clear_test_flags=True):
    """Human exception review (H1) then resume from the right step using the saved state."""
    state = load_state(claim_id)
    if clear_test_flags:
        for k in ("force_bad_json", "force_api_fail", "force_model_fail"):
            state["config"][k] = False
    action = {"ts": now(), "reviewer": reviewer, "note": note, "previous_status": state["status"]}
    print(f"\n=== Resume {claim_id} by {reviewer} (was {state['status']} at {state['current_step']}) ===")
    if new_input:                                   # customer supplied missing info -> restart at S1
        state["input"].update(new_input)
        action["action"], start = f"added missing fields {list(new_input)}", "S1"
    elif corrected_fields:                          # reviewer corrected extraction -> continue at S5
        base = (state["results"].get("S3") or {}).get("extracted") or {}
        base.update(corrected_fields)
        state["results"]["S3"] = {"extracted": base, "attempts": [], "corrected_by": reviewer}
        state["results"]["S4"] = {"issues": [], "overridden_by": reviewer}
        action["action"], start = f"corrected fields {list(corrected_fields)}", "S5"
    elif override_coverage:                         # reviewer confirms coverage -> continue at S6
        state["results"]["S5"]["result"] = "COVERED"
        state["results"]["S5"]["reasons"].append(f"Coverage confirmed by reviewer {reviewer}.")
        action["action"], start = "coverage confirmed", "S6"
    else:                                           # plain restart of the step that stopped
        action["action"], start = "retry from last step", state["current_step"]
    state["human_actions"].append(action)
    log_step(state, "H1", "human", f"{reviewer}: {action['action']} - {note}")
    return run_from(state, start)
```

Figure E. Human exception review and resume. The reviewer action is recorded and the run restarts at S1, the stopped step, S5 or S6.